

FINAL YEAR PROJECT REPORT

Little Share – Micro Donation App

A Progressive Web Application for Community-Based Micro Donations

Academic Year 2025–2026

1. Project Overview

Little Share is a Progressive Web Application (PWA) designed to facilitate micro-level donations within local communities. The platform connects donors who have surplus items—such as clothing, school supplies, electronics, and other household goods—with receivers who genuinely need them. Unlike traditional charity platforms that deal with monetary donations, Little Share focuses exclusively on in-kind donations, enabling a direct and personal giving experience.

The application is built using modern web technologies including React 18 with TypeScript for the frontend, Supabase as the Backend-as-a-Service (BaaS) for authentication, database management, real-time subscriptions, file storage, and serverless edge functions. The entire application is designed as a mobile-first PWA, meaning users can install it on their Android, iOS, or desktop devices and use it like a native application with offline capabilities, push notifications, and home screen access.

The core philosophy behind Little Share is that even the smallest act of giving—a used school bag, a winter jacket, or an old smartphone—can make a significant difference in someone's life. The platform removes the friction of traditional donation processes by providing a seamless digital experience where donors can list items in seconds and receivers can browse, request, and coordinate pickup with just a few taps.

The system supports three user roles: Donor (users who wish to give items), Receiver (users who need items), and Both (users who participate in both activities). This flexibility ensures that the platform remains inclusive and community-driven, where today's receiver can become tomorrow's donor.

2. Problem Statement

In many communities, individuals possess surplus items that are still in usable condition but are often discarded or left unused. Simultaneously, there are underprivileged individuals and families who lack access to basic necessities such as clothing, school supplies, and electronics. The disconnect between those who have surplus and those who are in need creates a significant social gap.

Traditional donation methods suffer from several limitations:

Lack of visibility: People willing to donate often do not know who needs their items or how to reach them effectively.

Trust deficit: Both donors and receivers lack a trusted platform to connect, verify identities, and communicate safely.

Geographical barriers: Even when willing, donors and receivers may not be aware of each other's proximity, leading to logistical challenges.

No real-time communication: Existing platforms do not offer real-time chat, location sharing, or AI-powered matching to facilitate smooth item transfers.

Absence of AI-driven selection: When multiple people request the same item, there is no intelligent mechanism to identify the most genuine need.

Little Share addresses all of these challenges by providing a comprehensive digital platform with real-time capabilities, location-aware features, AI-powered request analysis, and a user-friendly mobile experience.

3. Objectives of the Project

The primary objectives of the Little Share project are:

- To develop a user-friendly Progressive Web Application that enables community members to donate and receive items with minimal friction.
- To implement a secure authentication system with email verification, password reset, and session management using Supabase Auth.
- To provide a comprehensive item listing system where donors can upload multiple images, specify item categories, condition, pickup location, and urgency status.
- To create a request management system where receivers can request items and donors can accept or decline requests with full visibility.
- To integrate real-time chat functionality with text messages, image sharing, location sharing, and live location tracking between matched donors and receivers.
- To implement an interactive map view using Leaflet.js and OpenStreetMap/Google Maps tiles, allowing users to discover nearby donations with distance-based filtering.
- To incorporate AI-powered request analysis using Supabase Edge Functions, which evaluates and ranks multiple requests for the same item based on emotional intensity, urgency, clarity, and specificity.
- To implement push notifications using Web Push API and VAPID keys, ensuring users are informed about new messages, request updates, and item availability changes in real time.
- To deliver a fully installable PWA experience with service worker support, offline capabilities, and native-like performance across Android, iOS, and desktop platforms.
- To ensure data security through Row-Level Security (RLS) policies on all database tables, ensuring users can only access data they are authorized to view or modify.

4. Features of the System

4.1 User Authentication and Registration

The application implements a complete authentication system powered by Supabase Auth. Users can register with their full name, email address, mobile number, password, user type (donor/receiver/both), and optional location details. The registration form includes real-time form validation using Zod schema validation, password strength indicators, and automatic geolocation detection using the browser's Geolocation API combined with OpenStreetMap's Nominatim reverse geocoding service. After registration, users receive a confirmation email with a verification link. The login system supports email/password authentication with a "Forgot Password" flow that sends a reset link to the user's email.

4.2 Dashboard

The dashboard is the central hub of the application. It displays a personalized greeting with the user's first name, a search bar for finding items, category cards showing the count of available items in each category (Clothing, School Supplies, Electronics, Other), an "Urgent Needs" section highlighting items marked as urgent, a "Nearby Donations" section, and a "Recently Added" section. The dashboard supports real-time updates through Supabase's Postgres Changes subscriptions, meaning new items appear automatically without page refresh. It also features a notification badge showing unread notification count, dark/light theme toggle, and pull-to-refresh functionality for mobile users.

4.3 Item Donation (Add Item)

Donors can list items for donation through a rich form interface. The form supports uploading up to 5 images per item with the first image designated as the cover photo. Each image is uploaded to Supabase Storage's "item-images" bucket. Donors specify the item name, category (Clothing, School Supplies, Electronics, Other), condition (New, Like New, Good, Fair), description, pickup address (with auto-detection using GPS and reverse geocoding), contact number (pre-filled from profile), and an urgency toggle. Upon successful submission, the donor is presented with a celebration screen featuring motivational quotes and confetti-like animations.

4.4 Browse and Search

The Browse page allows receivers to discover available items through category-based filtering and search. Items are displayed as cards showing the item image, name, category badge, condition, donor name, and time since listing. Users can filter by specific categories and search by item name or category keywords. Each item card is clickable, leading to a detailed Item Detail page where the full description, all images, pickup location, and a "Request Item" button are displayed.

4.5 Request Management

The Requests page features a tabbed interface with two views: Incoming Requests (for donors) and My Requests/Outgoing (for receivers). Donors can see all requests for their listed items, with requester details including name, message, and mobile number. They can accept or decline requests. When a request is accepted, the item is automatically marked as unavailable, and all other pending requests for that item are automatically declined with notifications sent to those requesters. Accepted requests unlock the chat functionality and display the requester's contact number. The system also includes AI-powered request analysis for items with multiple pending requests.

4.6 Real-Time Chat System

The chat system is implemented as a dialog-based interface that opens when donors and receivers are matched (request accepted). It supports text messages with optimistic UI updates (messages appear instantly before server confirmation), image sharing via Supabase Storage's "chat-media" bucket, static location sharing using GPS coordinates, and live location sharing with continuous GPS tracking. The chat displays date separators, read receipts (seen status), typing indicators, and online/offline presence status. Messages are stored in the "messages" table with real-time subscriptions for instant delivery. The system also supports a full-screen map preview showing shared locations and live user positions.

4.7 Interactive Map View

The Map page provides a visual, location-based discovery experience using Leaflet.js with React-Leaflet. It displays donation items as custom category-themed markers on the map. Users can filter items by distance (1km, 5km, 10km, 50km) from their current location, toggle between street view and satellite imagery (using Google Maps tiles), switch between map and list views, and recenter the map on their current location. The user's position is shown as an animated blue dot (Google Maps style) with an accuracy circle. The map implements the Haversine formula for accurate distance calculations and supports both high-accuracy GPS (mobile) and network-based (desktop) positioning with automatic fallback.

4.8 AI-Powered Request Analysis

When an item receives multiple pending requests, donors can invoke an AI analysis feature. This calls a Supabase Edge Function ("analyze-requests") that evaluates each request message based on four criteria: Emotional Intensity (0–30 points), Urgency (0–30 points), Clarity (0–20 points), and Specificity (0–20 points). The AI generates a composite score out of 100, provides a brief summary for each request, and ranks them from most to least genuine need. The top-ranked requester is highlighted as the "AI Recommended" choice. This feature helps donors make informed decisions when multiple people need the same item.

4.9 Push Notifications

The application implements Web Push Notifications using the VAPID (Voluntary Application Server Identification) protocol. A Supabase Edge Function ("get-vapid-key") serves the public VAPID key, and another function ("send-push-notification") handles sending push notifications to subscribed users. Push subscriptions are stored in the "push_subscriptions" table with endpoint, p256dh key, and auth secret. Users also receive in-app real-time notifications via Supabase's Postgres Changes, displayed as toast messages. The notification system covers new messages, request acceptances, request declines, and other important events.

4.10 PWA Installation

Little Share is a fully installable Progressive Web Application. The "/install" page provides detailed, device-specific installation instructions for Android (Chrome and Samsung Internet), iOS (Safari), and Desktop (Chrome/Edge). The app captures the browser's "beforeinstallprompt" event to enable a native install button. Once installed, the app runs in standalone mode with its own window, splash screen, and app icon. The PWA configuration includes a web manifest (manifest.json) and a service worker (sw.js) for caching and offline support.

4.11 Profile Management

Users can view and edit their profiles, including full name, mobile number, address, city, and avatar. Profile avatars are uploaded to Supabase Storage's "avatars" bucket. The profile system also supports location updates with GPS detection and manual entry. The settings page provides access to profile editing, notification preferences, privacy policy, help/FAQ, theme toggle, app installation, and logout functionality.

4.12 Notification Preferences

Users can customize their notification preferences through a dedicated settings interface. They can toggle notifications for new items, new requests, and request updates. They can also select preferred categories to receive notifications only for items in categories they care about. These preferences are stored in the “notification_preferences” table.

5. Technologies Used

Category	Technologies
Frontend Framework	React 18 with TypeScript 5, Vite 5 (build tool)
UI Library	Tailwind CSS v3, shadcn/ui (Radix UI primitives), Framer Motion (animations)
State Management	React Hooks (useState, useEffect, useCallback, useMemo), TanStack React Query v5
Routing	React Router DOM v6 with animated route transitions (AnimatePresence)
Backend (BaaS)	Supabase – Authentication, PostgreSQL Database, Realtime Subscriptions, Storage, Edge Functions (Deno)
Database	PostgreSQL (managed by Supabase) with Row-Level Security (RLS)
Maps & Geolocation	Leaflet.js with React-Leaflet, OpenStreetMap Nominatim (geocoding), Google Maps tiles
AI / ML	Supabase Edge Function calling AI API for request analysis and ranking
Push Notifications	Web Push API with VAPID protocol, Service Workers
Form Validation	Zod schema validation, React Hook Form
Date Handling	date-fns library
Charts	Recharts (data visualization library)
Theme Support	next-themes (dark/light mode with system preference detection)
PWA Support	Web App Manifest, Service Worker, beforeinstallprompt API
File Storage	Supabase Storage (avatars, item-images, chat-media buckets)
Deployment	Lovable Platform with automatic CI/CD

6. System Architecture

Little Share follows a modern three-tier client-server architecture optimized for real-time web applications:

6.1 Presentation Layer (Frontend)

The frontend is a single-page application (SPA) built with React 18 and TypeScript, bundled using Vite 5. The UI is composed of reusable components built with shadcn/ui (based on Radix UI primitives) and styled with Tailwind CSS. Animations and page transitions are handled by Framer Motion. The application uses React Router DOM v6 for client-side routing with AnimatePresence for smooth page transitions. State management is handled through React's built-in hooks and TanStack React Query for server state synchronization and caching.

6.2 Application Layer (Backend Services)

The backend is entirely managed by Supabase, a Backend-as-a-Service platform built on top of PostgreSQL. Supabase provides: Authentication Service (email/password auth with email verification, password reset, and session management via JWT tokens), Database Service (PostgreSQL with auto-generated REST API and real-time capabilities via WebSocket connections), Storage Service (S3-compatible object storage for avatars, item images, and chat media), Edge Functions (serverless Deno functions for AI request analysis, VAPID key management, and push notification delivery), and Realtime Engine (Postgres Changes for live data subscriptions and Presence for online status tracking).

6.3 Data Layer (Database)

The PostgreSQL database contains 9 tables with comprehensive Row-Level Security policies. All tables use UUID primary keys with automatic generation. Foreign key relationships enforce referential integrity between profiles, items, requests, messages, and related tables. Database triggers handle automatic timestamp updates and new user profile creation. The database also includes custom functions like "item_donor_id" for secure donor identification and "notify_new_message" for automated notification creation on new messages.

6.4 Communication Flow

The frontend communicates with the backend through: Supabase JavaScript SDK (@supabase/supabase-js) for REST API calls, authentication, and storage operations; WebSocket connections for real-time data subscriptions (new items, messages, notifications, presence); HTTPS calls to Supabase Edge Functions for AI analysis and push notification operations; and the Web Push API for delivering push notifications to subscribed devices via VAPID-authenticated requests.

7. Modules Description

7.1 User Module

Handles all user-related functionality including registration (with user type selection: donor/receiver/both), email verification, login, password reset, profile management (name, mobile, address, city, avatar, location coordinates), and session management. The module uses Supabase Auth for secure authentication with JWT tokens. A database trigger ("handle_new_user") automatically creates a profile entry in the public.profiles table when a new user registers through auth.users.

7.2 Donation Module (Item Management)

Manages the complete lifecycle of donated items. Donors can create listings with multiple images (up to 5), categorize items (Clothing, School Supplies, Electronics, Other), specify condition (New, Like New, Good, Fair), add descriptions, set pickup locations with GPS detection, provide contact numbers, and mark items as urgent. Items can be browsed, searched, and filtered by category. When a request is accepted, the item is automatically marked as unavailable.

7.3 Request Module

Handles the request workflow between donors and receivers. Receivers can send requests for available items with an optional message. Donors see incoming requests with requester details and can accept or decline them. The module implements exclusive matching: when one request is accepted, all other pending requests for the same item are automatically declined. The module also integrates with the AI analysis system and the notification module for real-time status updates.

7.4 Chat Module

Provides real-time communication between matched donors and receivers. The module supports text messages, image sharing (with Supabase Storage), static location sharing, and live location tracking with continuous GPS updates. It includes online/offline presence detection, read receipts, typing indicators, date separators, and an interactive map preview for shared locations. Messages are delivered in real-time via Supabase's Postgres Changes subscriptions with optimistic UI updates for instant feedback.

7.5 Map Module

Implements the interactive map view using Leaflet.js with React-Leaflet. The module displays donation items as custom category-themed markers, supports distance-based filtering (1–50km), street/satellite view toggle, list view toggle, real-time GPS tracking with accuracy indicators, and animated fly-to transitions. It uses the Haversine formula for distance calculations and supports both high-accuracy GPS and network-based positioning with automatic fallback.

7.6 Notification Module

Manages both in-app and push notifications. In-app notifications are created by database triggers (e.g., “notify_new_message”) and client-side functions (e.g., “notifyRequestAccepted”, “notifyRequestDeclined”). Real-time notification delivery uses Supabase’s Postgres Changes to show toast messages instantly. Push notifications use the Web Push API with VAPID keys, handled by Supabase Edge Functions. Users can customize notification preferences for different event types and categories.

7.7 AI Analysis Module

Provides intelligent request ranking when items have multiple pending requests. The module calls the “analyze-requests” Supabase Edge Function, which evaluates each request message across four dimensions: emotional intensity, urgency, clarity, and specificity. Each dimension is scored independently, and a composite score out of 100 determines the ranking. The AI provides a brief summary for each request, helping donors make informed decisions about who receives their donated item.

8. Database Structure

The application uses a PostgreSQL database managed by Supabase with 9 main tables. All tables implement Row-Level Security (RLS) policies to ensure data access control.

8.1 profiles

Stores user profile information. Primary key: id (UUID, references auth.users). Fields: full_name (text, required), mobile_number (text, required), user_type (enum: donor/receiver/both, default: both), address (text, nullable), city (text, nullable), avatar_url (text, nullable), latitude (double precision, nullable), longitude (double precision, nullable), created_at (timestamp), updated_at (timestamp).

8.2 items

Stores donated item listings. Primary key: id (UUID, auto-generated). Fields: donor_id (UUID, references profiles), name (text, required), description (text, nullable), category (enum: clothing/school_supplies/electronics/other), condition (enum: new/like_new/good/fair, default: good), image_url (text, nullable – comma-separated URLs for multiple images), pickup_address (text, nullable), pickup_latitude (double precision, nullable), pickup_longitude (double precision, nullable), contact_number (text, nullable), is_available (boolean, default: true), is_urgent (boolean, default: false), created_at (timestamp), updated_at (timestamp).

8.3 requests

Stores item requests from receivers. Primary key: id (UUID, auto-generated). Fields: item_id (UUID, references items), receiver_id (UUID, references profiles), message (text, nullable), status (enum: pending/accepted/declined/completed, default: pending), created_at (timestamp), updated_at (timestamp). RLS ensures only the receiver and item donor can view/update requests.

8.4 messages

Stores chat messages between donors and receivers. Primary key: id (UUID, auto-generated). Fields: request_id (UUID, references requests), sender_id (UUID, references profiles), content (text, required), message_type (text, default: text – supports text/image/location/live_location), media_url (text, nullable – for image messages), location_data (JSONB, nullable – stores latitude/longitude for location messages), is_read (boolean, default: false), created_at (timestamp).

8.5 notifications

Stores user notifications. Primary key: id (UUID, auto-generated). Fields: user_id (UUID, references profiles), title (text, required), message (text, required), type (text, required – e.g., new_message/request_accepted/request_declined), related_item_id (UUID, nullable, references items), related_request_id (UUID, nullable, references requests), is_read (boolean, default: false), created_at (timestamp).

8.6 Other Tables

`notification_preferences`: Stores per-user notification toggle settings and preferred categories.
`push_subscriptions`: Stores Web Push subscription data (endpoint, p256dh, auth) for delivering push notifications.
`chat_presence`: Tracks online/offline status and last seen time for chat participants.
`live_locations`: Stores real-time GPS coordinates for live location sharing during chats, with sharing status and expiration time.

8.7 Database Functions and Triggers

`handle_new_user()`: A trigger function that automatically creates a profile in `public.profiles` when a new user registers in `auth.users`.
`handle_updated_at()`: A trigger function that automatically updates the `updated_at` timestamp on row modifications.
`item_donor_id()`: A security definer function that returns the `donor_id` for a given item, used in RLS policies to verify donor identity without exposing the `items` table directly.
`notify_new_message()`: A trigger function that creates a notification entry whenever a new message is inserted, determining the recipient based on the request's donor/receiver relationship.

9. Workflow of the Application

9.1 Registration and Login Flow

Step 1: User opens the app and lands on the splash screen, which redirects to the Auth page. Step 2: New users select the “Register” tab and fill in their details (name, email, mobile, password, user type). They can optionally detect their location via GPS or pick it on a map. Step 3: On submission, Supabase Auth creates the account and sends a verification email. The user is redirected to the Email Confirmation page. Step 4: After clicking the verification link in their email, the user is redirected back to the app and logged in automatically. Step 5: Returning users can log in with email/password or use the Forgot Password flow.

9.2 Donation Flow

Step 1: A logged-in donor navigates to the “Add Item” page from the bottom navigation. Step 2: The donor fills in item details, uploads photos, selects category and condition, adds pickup location (auto-detected or manual), and optionally marks the item as urgent. Step 3: On submission, images are uploaded to Supabase Storage and the item record is created in the database. Step 4: The donor sees a celebration screen with a motivational quote. Step 5: The item immediately appears on the dashboard, browse page, and map for all users via real-time subscriptions.

9.3 Request and Matching Flow

Step 1: A receiver browses available items on the Dashboard, Browse page, or Map. Step 2: They open an item’s detail page and tap “Request Item” with an optional message. Step 3: The donor receives a notification and sees the request in their “Incoming Requests” tab. Step 4: If multiple requests exist for the same item, the donor can use AI Analysis to rank requests by genuine need. Step 5: When the donor accepts a request, the item is marked unavailable, other pending requests are auto-declined with notifications, and the chat functionality is unlocked for the matched pair.

9.4 Communication Flow

Step 1: After a request is accepted, both donor and receiver can access the chat via the “Chat” button on the request card. Step 2: Users can exchange text messages, share images, send their current location, or start live location sharing. Step 3: Messages appear in real-time with read receipts and online/offline status indicators. Step 4: Location messages can be tapped to open a full-screen interactive map showing all shared locations. Step 5: The donor’s contact number is also displayed for direct phone communication.

10. Special Features

10.1 AI-Powered Request Analysis

The AI analysis feature is a unique differentiator. When an item receives multiple requests, a Supabase Edge Function analyzes each request’s message using natural language processing criteria. The system scores emotional intensity (how deeply the person conveys their need), urgency (time-sensitive indicators), clarity (how well-articulated the request is), and specificity

(how specifically the request relates to the item). Results are presented with detailed score breakdowns and AI-generated summaries, empowering donors to make data-driven compassionate decisions.

10.2 Real-Time Everything

The application is built with real-time as a core principle. New items appear instantly on all users' dashboards. Chat messages are delivered and displayed in milliseconds with optimistic updates. Notification badges update in real-time. Online/offline presence is tracked live. Live location sharing provides continuous GPS updates. All of this is powered by Supabase's Realtime engine using WebSocket connections and Postgres Changes.

10.3 Interactive Map with GPS

The Leaflet.js-based map provides a Google Maps-like experience with custom category markers, distance-based filtering, satellite view toggle, animated user location dot with accuracy circle, smooth fly-to animations, and Haversine distance calculations. It supports both high-accuracy GPS (mobile) and network-based positioning (desktop) with automatic fallback.

10.4 Progressive Web App

Little Share is fully installable as a PWA. It includes a web manifest for app identity, a service worker for caching and offline support, the beforeinstallprompt API for native install prompts, standalone display mode for a native app experience, and a dedicated install page with device-specific instructions for Android, iOS, and Desktop.

10.5 Push Notifications

Real browser push notifications using the Web Push API with VAPID authentication ensure users stay informed even when the app is closed. The notification system covers new messages, request status changes, and other important events, delivered through Supabase Edge Functions that manage subscription storage and notification dispatch.

11. Advantages of the System

- Cross-platform: Works on Android, iOS, and Desktop as a PWA without requiring separate native app development.
- Real-time communication: Instant messaging, live location sharing, and presence detection create a modern, responsive user experience.
- AI-powered decision support: Intelligent request ranking helps donors allocate items to those with the most genuine need.
- Location-aware: GPS-based item discovery with interactive maps and distance filtering makes finding nearby donations effortless.
- Secure: Row-Level Security policies on every table ensure users can only access authorized data. Authentication uses industry-standard JWT tokens.
- Scalable: Supabase's managed PostgreSQL database and serverless Edge Functions scale automatically with user demand.

- Free to use: The platform facilitates in-kind donations with no monetary transactions, reducing barriers to participation.
- Modern UI/UX: Mobile-first design with dark/light themes, animations, pull-to-refresh, and staggered list transitions provides a polished experience.
- Privacy-conscious: Users control what information they share, with notification preferences and location sharing toggles.
- Community-driven: The dual donor/receiver role system fosters a sense of community where everyone can both give and receive.

12. Limitations

- Internet dependency: The application requires an active internet connection for core functionality including authentication, data synchronization, and real-time features. Offline support is limited to basic caching via the service worker.
- No monetary donation support: The platform is designed exclusively for in-kind donations and does not support monetary transactions or payment processing.
- Limited AI accuracy: The AI request analysis depends on the quality and length of request messages. Short or vague messages may not be scored accurately.
- iOS PWA limitations: iOS imposes restrictions on PWAs including limited push notification support (only available in iOS 16.4+), no background sync, and reduced storage allocation.
- No delivery/logistics system: The platform facilitates connections between donors and receivers but does not provide built-in delivery or logistics coordination beyond location sharing.
- Single language: The application currently supports only English. Multi-language support would be needed for broader community adoption.
- No verification system: There is no formal verification of receiver need or donor item quality beyond the information provided in listings and request messages.
- Limited moderation: The platform lacks an administrative moderation panel for managing reported items, disputes, or inappropriate content.

13. Future Enhancements

- Multi-language support: Implement internationalization (i18n) to support regional languages and make the platform accessible to non-English speaking communities.
- Admin dashboard: Build a comprehensive admin panel for user management, content moderation, analytics, and platform oversight.
- Delivery integration: Partner with local delivery services or volunteer networks to facilitate item pickup and delivery logistics.
- Gamification: Introduce donor leaderboards, badges, donation streaks, and community impact scores to encourage continued participation.
- Image-based AI categorization: Use computer vision APIs to automatically detect item category, condition, and description from uploaded photos.

- Social sharing: Allow users to share item listings on social media platforms to increase visibility and community engagement.
- Monetary donation support: Integrate payment gateways like Stripe or Razorpay to enable optional monetary donations alongside in-kind giving.
- NGO partnerships: Create organization accounts for NGOs and charities to manage bulk donations, volunteer coordination, and impact reporting.
- Advanced matching algorithm: Develop a sophisticated matching system that considers user history, location proximity, need verification, and community reputation.
- Offline-first architecture: Implement full offline support with IndexedDB for local data storage and background sync for seamless operation without internet.

14. Conclusion

Little Share – Micro Donation App represents a comprehensive solution to the challenge of connecting donors with receivers at the community level. By leveraging modern web technologies including React 18, TypeScript, Supabase, Leaflet.js, and AI-powered analysis, the application delivers a feature-rich, real-time, and user-friendly platform for facilitating in-kind donations.

The Progressive Web Application approach ensures cross-platform compatibility without the overhead of native app development, while the real-time architecture powered by Supabase's Realtime engine provides an engaging and responsive user experience. The AI-powered request analysis feature adds a unique dimension of intelligent decision support that sets Little Share apart from conventional donation platforms.

With its secure authentication, comprehensive Row-Level Security policies, interactive map-based discovery, real-time chat with multimedia support, and push notification system, Little Share demonstrates how modern web technologies can be combined to create a socially impactful application that bridges the gap between those who have surplus and those who are in need.

The platform has significant potential for future growth through multi-language support, NGO partnerships, delivery integration, and advanced AI-based matching algorithms. Little Share is not just a technical project – it is a step towards building more connected, compassionate communities where every small act of sharing can make a meaningful difference.